

Power BI Best Practices and Guidelines

Authors	Date	Version
Derek	2026-01-07	1.1

This guide outlines best practices for Power BI data modeling, DAX optimization, and report design to ensure high performance, maintainability, and usability.

Contents

- Practice Roadmap
 - Suggested Learning Pace
 - Learning Tips
- Data Modeling
- DAX Best Practices
 - Before You Start
 - Improving DAX Syntax
 - Optimizing DAX Functions
 - Common Mistakes to Avoid
 - Frequently Used DAX Functions
 - References
- Power BI Reports Best Practices
 - General Guidelines
 - Visual Use Cases and Examples
- Frequent Asked Questions

Practice Roadmap

- Integrate modeling, DAX, and reporting exercises on a shared sample dataset; complete a minimal star schema, core measures, and a one-page report within a single sprint to keep context aligned.
- Iterate on the same model with performance tuning, DAX refactoring, and new visuals, validating each change with queries, aggregations, or performance counters.
- Publish a prototype, enable security or incremental refresh, and collect stakeholder feedback to confirm usability and performance goals.

Suggested Learning Pace

Phase	Focus	Estimated Duration
Foundation	Assemble a star model, baseline measures, and a focused report page.	2–3 weeks
Optimization	Tune storage, refine DAX patterns, and streamline visual interactions.	3–4 weeks

Phase	Focus	Estimated Duration
Mastery	Apply governance, automate refresh, and document reusable patterns.	4+ weeks

Learning Tips

- Use real scenarios or reputable public datasets so practice translates to production habits.
- Revisit each section after hands-on work to connect concepts with observed results.
- Track effort across modeling, DAX, and visualization to target future study.

Data Modeling

1. Import only necessary fields and tables instead of entire datasets. Ensure the model is as narrow and lean as possible. Power BI uses a columnar storage engine, so longer and leaner tables perform better. For very large tables, consider partitioning and processing partitions in parallel.

Example (SQL):

```
SELECT OrderID, OrderDate, CustomerKey, ProductKey, SalesAmount
FROM dbo.FactSales;
```

Example (Power Query M):

```
Source = Sql.Database("server", "db"),
FactSales = Source{[Schema="dbo",Item="FactSales"]}[Data],
Narrow = Table.SelectColumns(FactSales,
{"OrderID","OrderDate","CustomerKey","ProductKey","SalesAmount"})
```

2. Limit complex measures and aggregations in the model. Prefer calculated measures over calculated columns. Where possible, push calculations upstream to the data source; closer to source generally performs better and centralizes logic.

Example (measure vs. calculated column):

```
-- Good (measure)
Total Sales = SUM( FACT_Sales[SalesAmount] )

-- Good (running total measure)
Running Total Sales =
    CALCULATE(
        [Total Sales],
        FILTER(ALLSELECTED(DIM_Date[Date]), DIM_Date[Date] <=
MAX(DIM_Date[Date]))
    )
```

```
-- Avoid (calculated column; inflates storage, ignores filter context  
at query time)  
Row Total (column) = FACT_Sales[Qty] * FACT_Sales[UnitPrice]
```

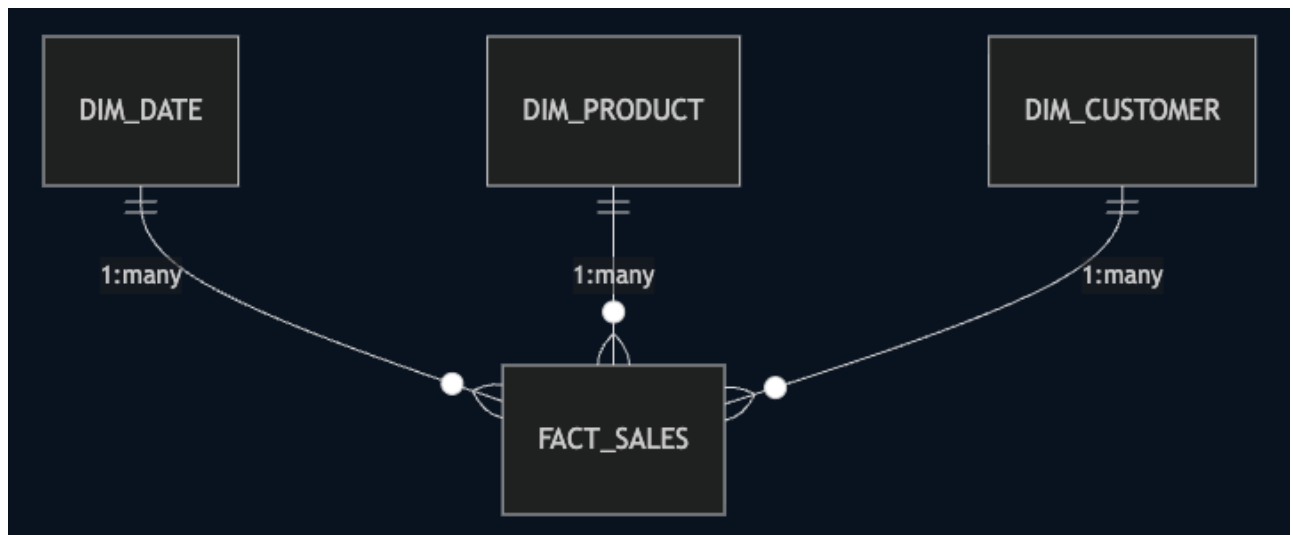
3. Prefer a star schema over a snowflake schema when possible. Snowflake schemas increase query complexity and change management. Star schemas are easier to read, use fewer joins, and reduce redundancy.

Example (star schema grain):

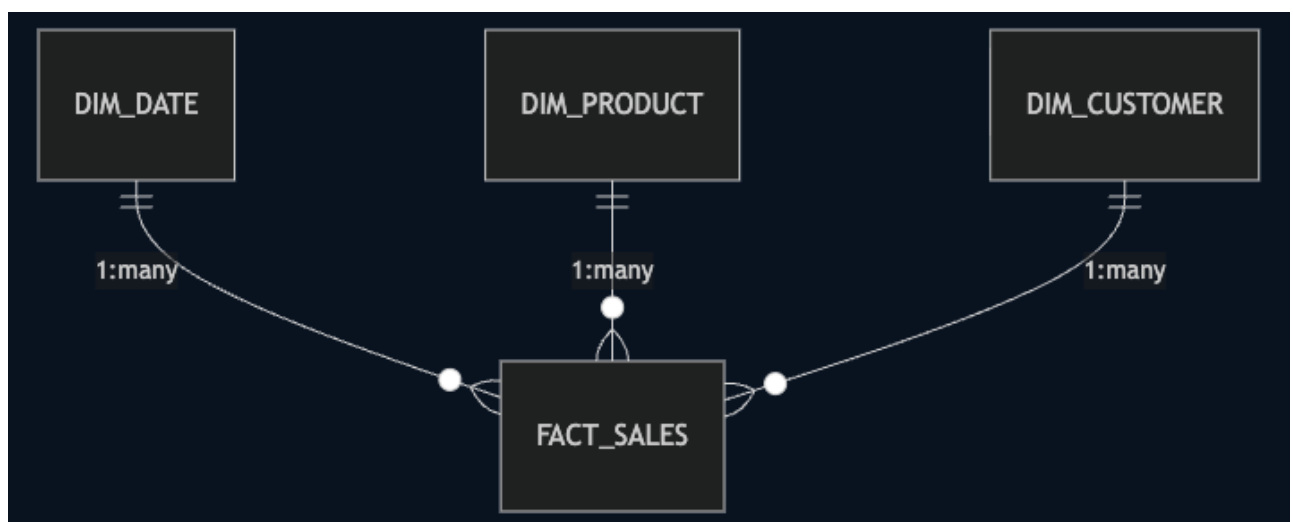
- `FACT_Sales(OrderDateKey, ProductKey, CustomerKey, SalesAmount, Qty)`
- `DIM_Date(DateKey, Date, Year, Month, Quarter)`
- `DIM_Product(ProductKey, Product, Category, Subcategory)`
- `DIM_Customer(CustomerKey, Customer, Segment, Region)`

Model relationships: `FACT_Sales[ProductKey] → DIM_Product[ProductKey]` (Many-to-One, single-direction), etc.

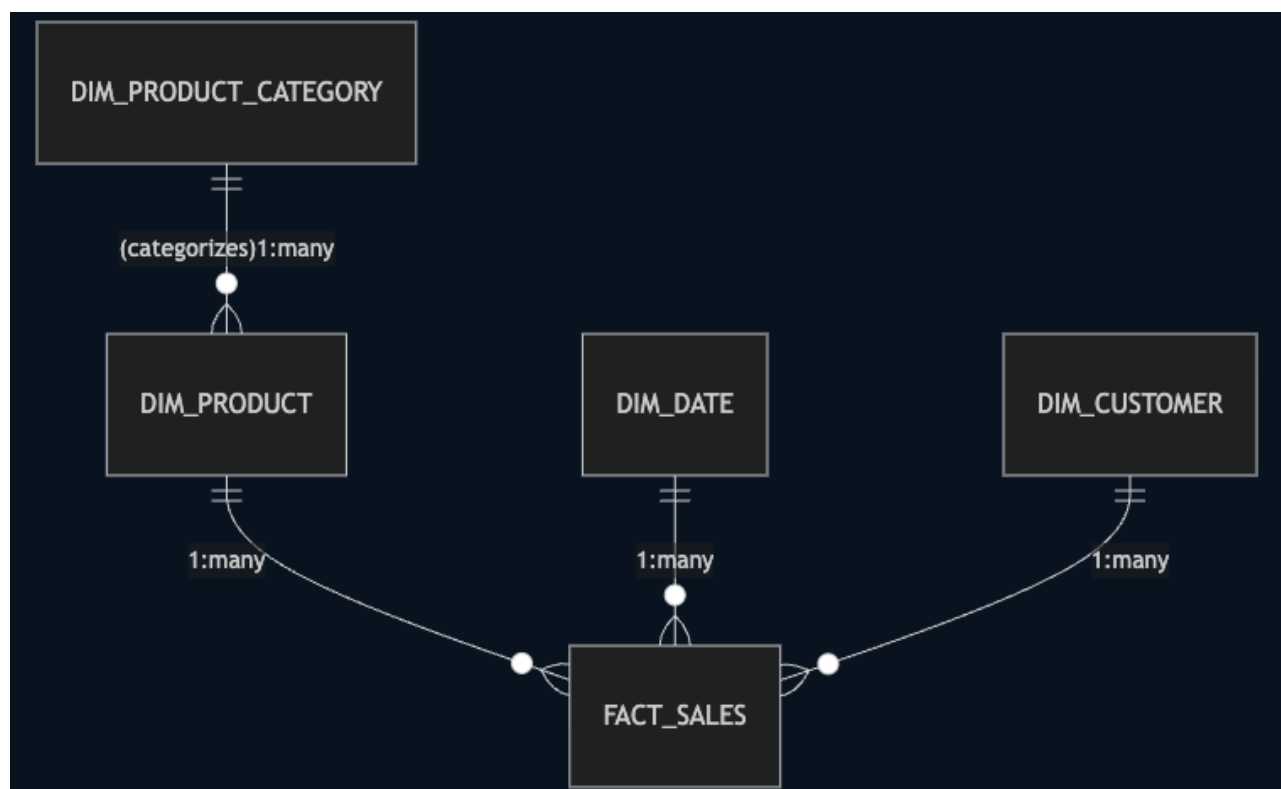
ER Diagram with cardinality



Star Schema



Snowflake Schema



4. Avoid bi-directional and many-to-many relationships on high-cardinality columns. These relationships increase filter paths and checks, often harming report performance.

Example (keep relationships single-direction; override in a measure only when needed):

```

Sales (Respect Date) =
    CALCULATE(
        [Total Sales],
        CROSSFILTER( DIM_Date[DateKey], FACT_Sales[OrderDateKey], Both
    )
)

-- Or use inactive relationship for ship date analysis
Sales (Ship Date) =
    CALCULATE([Total Sales], USERELATIONSHIP(DIM_Date[DateKey],
FACT_Sales[ShipDateKey]))
  
```

CROSSFILTER adjusts the active relationship direction between **DIM_Date[DateKey]** and **FACT_Sales[OrderDateKey]**

5. Avoid floating-point data types where practical. Floating-point can introduce rounding issues and increase memory. Prefer integers or fixed-decimal types for measures that require precision.
- Tip: In the Modeling pane, use the **Decimal number** (fixed decimal via Currency) or **Whole number** data types for financial and counting measures.
 - Avoid **Floating-point** types in sources when exact precision matters.

6. Use clear naming conventions. For example, name fact tables **FACT_<TableName>** and dimension tables **DIM_<TableName>**.
7. Rename fields to business-friendly terms. Standardize names across the business to reduce per-visual renaming.
8. Split very long text fields into smaller attributes when appropriate.
9. Prefer numeric keys for relationships over string keys to reduce dictionary size and improve compression.
10. Reduce datetime precision when possible. Use **Date** instead of **DateTime**, or split into separate **Date** and **Time** columns if needed.

Example (Power Query M – split DateTime):

```
let
    Source = ...,
    WithDate = Table.AddColumn(Source, "OrderDate", each
        Date.From([OrderDateTime]), type date),
    WithTime = Table.AddColumn(WithDate, "OrderTime", each
        Time.From([OrderDateTime]), type time),
    RemoveOriginal = Table.RemoveColumns(WithTime, {"OrderDateTime"})
in
    RemoveOriginal
```

11. Assign correct data types and formats. Ensure numeric, date, and boolean columns use appropriate types.
12. Use data preview and profiling to validate inputs.
 - In Power Query, enable View > Column quality/distribution/profile to surface errors and data composition before loading.
13. Avoid using long, high-cardinality columns in slicers and as relationship keys.
14. Disable Auto Date/Time and use a dedicated Calendar table. Auto Date/Time creates hidden tables that bloat models. Instead, add a reusable calendar with the attributes you need.
 - [Creating a Calendar in Power BI](#)

To disable Auto Date/Time: File > Options and settings > Options > Global or Current File > Data Load > Time intelligence > Auto Date/Time.

Example (Calendar table DAX)

```
Calendar =
    ADDCOLUMNS(
        CALENDAR( DATE(2019,1,1), DATE(2030,12,31) ),
        "Year", YEAR([Date]),
```

```

    "Month", FORMAT([Date], "MMMM"),
    "Month No", MONTH([Date]),
    "Quarter", "Q" & FORMAT([Date], "Q"),
    "Year-Month", FORMAT([Date], "YYYY-MM"),
    "Week", WEEKNUM([Date])
)

```

After creating the table: Table tools > Mark as date table > select **Date**.

15. Avoid high-cardinality keys for joins. Prefer surrogate integer keys over text when possible.
16. Hide fields and tables not used by reports (e.g., unused foreign keys).
17. Remove unused relationships automatically created by Power BI.
18. Consider suffixing columns used for joins with **_key** for easy identification (e.g., **customer_id_key**).
19. Organize the model with a “waterfall” layout:
 - Place dimension tables at the top and fact tables at the bottom to reflect data flow.
 - Keep measure tables on the right; supporting/helper tables on the left.
 - Ensure relationships are visible to make issues easier to spot.
20. Plan for large datasets with a clear strategy:
 - Aggregate to the required grain in SQL or Power Query to reduce data size.
 - Remove unused fields to keep tables narrow.
 - Use dataflows for centralized ingestion and refresh.
 - Configure incremental refresh to append only new data.
 - Use **row-level security** to pre-filter data per user.
 - Consider DirectQuery and/or composite models to limit in-memory volume.
 - Split monolithic reports into smaller, purpose-built reports where appropriate.

Examples:

- SQL aggregation view (reduce grain before import):

```

CREATE VIEW dbo.vw_FactSalesDaily AS
SELECT
    CAST(OrderDate AS date) AS OrderDate,
    ProductKey,
    CustomerKey,
    SUM(SalesAmount) AS SalesAmount,
    COUNT(*) AS OrderCount
FROM dbo.FactSales
GROUP BY CAST(OrderDate AS date), ProductKey, CustomerKey;

```

- Power Query incremental refresh filter step (requires **RangeStart/RangeEnd** DateTime parameters):

```
Source = ...,
Filtered = Table.SelectRows(Source, each [OrderDate] >=
RangeStart and [OrderDate] < RangeEnd)
```

Then, in Desktop: Model view > Table > Incremental refresh > set "Store rows in the last ..." and "Incrementally refresh data starting ...".

- Simple RLS example (table filter):

```
-- On DIM_Salesperson
DIM_Salesperson[UPN] = USERPRINCIPALNAME()
```

- Composite model idea: Import small dimensions (Date, Product, Customer), keep the large fact table in DirectQuery; add Aggregations over Import to accelerate common queries.

DAX Best Practices

This guide helps you speed up reports by optimizing DAX.

Before You Start

- Clear your DAX cache before benchmarking changes. You can do this in DAX Studio.
- Use a formatter for readability, e.g., [DAX Formatter](#). Readable code is easier to review and optimize.

Improving DAX Syntax

- Use **DISTINCT()** and **VALUES()** consistently.
 - Power BI may add a blank due to referential integrity gaps (especially in DirectQuery).
 - **DISTINCT()** includes a blank only if it exists in source data.
 - **VALUES()** includes source blanks and blanks added by Power BI. Choose one approach and be consistent.
- Fully qualify columns; don't fully qualify measures.
 - With column refs: **Profit = Orders[Sales] - Orders[Cost]**
 - Without column refs: **Profit = [Sales] - [Cost]**
 - This eliminates ambiguity and avoids breakage when changing a measure's home table.

Optimizing DAX Functions

- Use **ISBLANK()** instead of **= BLANK()** comparisons.
 - **ISBLANK()** checks strictly for blanks; **= BLANK()** can also match empty strings depending on context.
- Use **= 0** instead of **ISBLANK() || = 0** when appropriate.

- In numeric comparisons, `BLANK()` behaves as zero; `= 0` covers both blank and zero. Use `[Value] IN { 0 }` to match only zero if needed.
- Prefer `SELECTEDVALUE()` over `HASONEVALUE() + VALUES()`.
 - `SELECTEDVALUE()` returns the single value or blank without additional error handling.
- Prefer `SELECTEDVALUE()` over `VALUES()` where multiple values are possible.
 - Avoid wrapping `VALUES()` with error functions for this purpose.
- Use variables to avoid repeated evaluation.

Incorrect:

```
Ratio = IF([Total Rows] > 10, DIVIDE(SUM(Revenue), [Total Rows]), 0)
```

Better:

```
Ratio =
    VAR totalRows = [Total Rows]
    RETURN IF(totalRows > 10, DIVIDE(SUM(Revenue), totalRows), 0)
```

- Use `DIVIDE()` instead of `/` when the denominator can be zero.
- Prefer `KEEPFILTERS()` over wrapping everything in `FILTER()` when maintaining current context is desired.
- Use `FILTER( ALL( Table[Column] ), Table[Column] = "Red" )` instead of filtering over `VALUES()` when you need to ignore existing filters on that column. Apply filters at the column level rather than the entire table when possible.
- Prefer `COUNTROWS()` over `COUNT()` when counting rows.

```
-- Column count (sensitive to blanks)
Sales Orders = COUNT( Sales[OrderDate] )

-- Clearer intent, ignores blanks
Sales Orders = COUNTROWS( Sales )
```

- Always pass the optional not-found parameter to `SEARCH()/FIND()` to avoid error branches.
- `ALLEXCEPT()` vs `ALL()`:
 - `ALLEXCEPT()` preserves filters only on specified columns (typically on the visual). It does not preserve filters on columns not present on the visual.

- Prefer `ALL()` when you intend to remove filters broadly; be explicit about the column or table scope.

Common Mistakes to Avoid

- Don't convert `BLANK()`s to zeros or strings without reason. Power BI efficiently filters blanks in visuals; replacing them can increase result set size and hurt performance.
- For ratios, prefer `(a - b) / b` with variables to avoid duplicate calculations and allow blanks to flow through (so rows with blanks can be filtered out), rather than `a/b - 1` or `a/b*100 - 100`.
- Avoid `IFERROR()/ISERROR()` patterns used in Excel.
 - Use the not-found parameter of `SEARCH()/FIND()` and functions like `DIVIDE()/SELECTEDVALUE()` that handle edge cases natively.
- Don't use scalar variables inside `SUMMARIZE()`; prefer `SUMMARIZECOLUMNS()` for groupings with filter context and measures. Use `SUMMARIZE()` only for pure grouping without aggregations, e.g., `SUMMARIZE( Table, Column1, Column2 )`.
- Avoid `ADDCOLUMNS()` in measures when it creates nested iterations; refactor to set-based expressions or precompute in Power Query.
- Convert two-valued columns to Boolean where possible; booleans and integers compress and scan faster.
- Avoid filtering on strings; prefer integer surrogate keys to leverage value encoding (works on integers).
- Push complex, repeated calculations upstream when feasible (ETL, views, or Power Query) to simplify DAX and improve performance.

Frequently Used DAX Functions

This quick-reference lists commonly used DAX functions with short examples. For performance guidance (e.g., `COUNTROWS` vs `COUNT`, `VALUES` vs `DISTINCT`, `DIVIDE`), see [DAX Best Practices](#).

Aggregation

- `SUM(column)` — totals a numeric column.

```
Total Sales = SUM( FACT_Sales[SalesAmount] )
```

- `COUNTROWS(table)` vs `COUNT(column)` — prefer `COUNTROWS` for row counts or when blanks exist.

```
Orders (rows) = COUNTROWS( FACT_Sales )
Orders (dates) = COUNT( FACT_Sales[OrderDate] )
```

- **AVERAGE(column), MIN(column), MAX(column)** — simple aggregations.

Filter & Context

- **CALCULATE(<measure>, <filters>...)** — changes filter context.

```
Sales US = CALCULATE( [Total Sales], DIM_Geography[Country] = "United States" )
```

- **FILTER(table, condition)** — returns a filtered table (use in **CALCULATE**/iterators).

```
Sales High Value =  
    CALCULATE( [Total Sales], FILTER( FACT_Sales,  
    FACT_Sales[SalesAmount] > 1000 ) )
```

Time Intelligence

- **SAMEPERIODLASTYEAR(dates)** — prior-year period.

```
Sales LY = CALCULATE( [Total Sales], SAMEPERIODLASTYEAR(  
    DIM_Date[Date] ) )  
YoY % = DIVIDE( [Total Sales] - [Sales LY], [Sales LY] )
```

- **DATEADD(dates, -1, MONTH)** — previous month (or other intervals).

```
Sales PM = CALCULATE( [Total Sales], DATEADD( DIM_Date[Date], -1,  
    MONTH ) )
```

- **TOTALYTD(<measure>, dates)** — year-to-date.

```
Sales YTD = TOTALYTD( [Total Sales], DIM_Date[Date] )
```

- **DATESINPERIOD(dates, anchor, -90, DAY)** — rolling window.

```
Sales 90D =  
    VAR Anchor = MAX( DIM_Date[Date] )  
    RETURN CALCULATE( [Total Sales], DATESINPERIOD( DIM_Date[Date],  
    Anchor, -90, DAY ) )
```

Table Expressions

- Prefer **SUMMARIZECOLUMNS** for groupings with measures; reserve **SUMMARIZE** for pure grouping (no measures).

```
Sales by Product =
    SUMMARIZECOLUMNS(
        DIM_Product[Product],
        "Sales", [Total Sales]
    )
```

- **ADDCOLUMNS**(table, name, expr, ...) — add calculated columns to a virtual table (avoid heavy use inside measures when it causes nested iteration).

```
Products with Margin =
    ADDCOLUMNS(
        VALUES( DIM_Product[Product] ),
        "Sales", [Total Sales],
        "Cost", [Total Cost],
        "Margin", [Total Sales] - [Total Cost]
    )
```

X-Functions (Row Iterators)

- **SUMX**(table, expr) — sum of row-wise expression.

```
Revenue = SUMX( FACT_Sales, FACT_Sales[Qty] * FACT_Sales[UnitPrice] )
```

- **AVERAGEX**(table, expr) — average of row-wise expression (e.g., weighted averages).

```
Avg Price (weighted) = DIVIDE( SUMX( FACT_Sales, FACT_Sales[Qty] *
    FACT_Sales[UnitPrice] ), SUM( FACT_Sales[Qty] ) )
```

- **RANKX**(table, expr, , order, ties) — ranking.

```
Product Rank by Sales = RANKX( ALL( DIM_Product[Product] ), [Total
    Sales], , DESC, Dense )
```

Other Essentials

- **ALL()** — remove filters to compute totals/shares.

```
Category Share = DIVIDE( [Total Sales], CALCULATE( [Total Sales], ALL(
DIM_Product[Category] ) ) )
```

- **VALUES()** vs **DISTINCT()** — both return unique values; **VALUES** may include Power BI-added blank. Use consistently across a report.

```
Selected Category = SELECTEDVALUE( DIM_Product[Category], "(All)" )
```

- **DIVIDE(numer, denom, 0)** — safe division; avoids divide-by-zero errors.
- **ISBLANK()** — explicit blank checks in conditions.

```
Safe % = IF( ISBLANK( [Sales LY] ), BLANK(), DIVIDE( [Total Sales],
[Sales LY] ) )
```

- **VAR ... RETURN** — store intermediate results; improves readability and performance.

```
Margin % =
    VAR s = [Total Sales]
    VAR c = [Total Cost]
    RETURN DIVIDE( s - c, s )
```

- Relationships in measures: **CROSSFILTER**, **USERELATIONSHIP**.

```
Sales (Ship Date) = CALCULATE( [Total Sales], USERELATIONSHIP(
DIM_Date[DateKey], FACT_Sales[ShipDateKey] ) )
```

References

- DAX Formatter: <https://www.daxformatter.com/>
- DAX Studio: <https://daxstudio.org/>
- SQLBI articles: <https://www.sqlbi.com/>

Power BI Reports Best Practices

General Guidelines

1. Avoid “data dump” pages with very wide tables.
2. Use slicers and filters to focus analysis, but keep them sparse. Avoid high-cardinality slicers, and use Query reduction Apply buttons to batch interactions. See also: Visual Use Cases — Slicers.

3. Use bookmarks and drill-through pages, and prefer drill-through buttons over right-click to make navigation discoverable. See also: Visual Use Cases — Drill-through Buttons, Tooltips.

Drill-through in Power BI enables report consumers to right-click a data point and navigate to a dedicated page filtered for that specific context, allowing deeper analysis of the selected entity without cluttering the main report canvas.

4. Prioritize end-user usability in layout, interactions, and navigation.
5. Give each page and visual a clear purpose.
6. Provide clear instructions and interaction cues for consumers.
7. Verify data accuracy and measure results under different filter contexts.
8. Use bookmarks and concise on-page text to guide users to interactions (e.g., drill-through, tooltips). See also: Visual Use Cases — Tooltips.
9. Review visuals for potential misinterpretation and clarify where needed.
10. Solicit consumer feedback on both presentation and data quality.
11. Be intentional in visual selection. Favor clarity over variety for its own sake.
12. Use deliberate placement and hierarchy to guide attention.
13. Keep placement of recurring elements consistent across pages.
14. Use layouts that support the story you intend to tell.
15. Use color sparingly and consistently, with clear semantic meaning.
16. Follow data viz best practices for palettes. If required to use vibrant brand colors, prefer muted variants in visuals.
17. Reduce clutter so consumers can focus on what matters.
18. Address accessibility early (color contrast, alt text, keyboard nav, etc.).
19. Include a survey link for feedback (not issue reporting). Also document how to get help or report issues (solution name, service offering) so tickets route correctly.
20. Prefer native visuals; use certified custom visuals only when necessary and performance-tested on your data. See also: Visual Use Cases — Decomposition Tree, Small Multiples (performance considerations).
21. Limit visuals per page. Too many visuals hurt performance. As a rule of thumb, limit a page to ~30 "points," where different visuals have approximate costs:
 - Cards: 1
 - Gauges: 2
 - Charts: 3
 - Maps: 3
 - Grids: 5 (limit to one per page)

- Also limit dashboards to ~10 tiles.
- 22. Remove unnecessary visual interactions. By default, visuals interact; disable interactions that don't add value to reduce backend queries.
- 23. Use templates (.pbix) to standardize and speed development. Templates can pre-apply themes, colors, common connections, and shared measures.
- 24. Reduce interactive queries via Query reduction settings. Add Apply buttons to slicers and the Filter pane to batch changes.
- 25. Reduce data loaded on landing pages. Use bookmarks, drill-through, and tooltips to defer heavy visuals until needed.
- 26. Use page backgrounds for static imagery instead of separate image visuals.
- 27. Use white or light backgrounds to improve printability and readability.
- 28. Test report performance on typical user devices and connections. Optimize for the lowest common denominator in your audience.

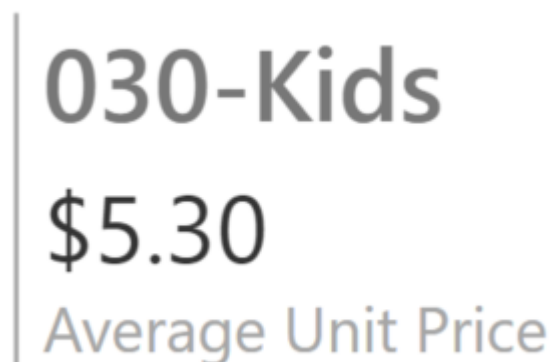
Visual Use Cases and Examples

- **Cards (single KPI)**

- Use for one key metric (e.g., Current Month Sales, On-time %).
- Example measures:

```
Total Sales = SUM( FACT_Sales[SalesAmount] )  
Sales Target = SUM( FACT_Targets[SalesTarget] )  
Sales vs Target % = DIVIDE([Total Sales] - [Sales Target], [Sales  
Target])
```

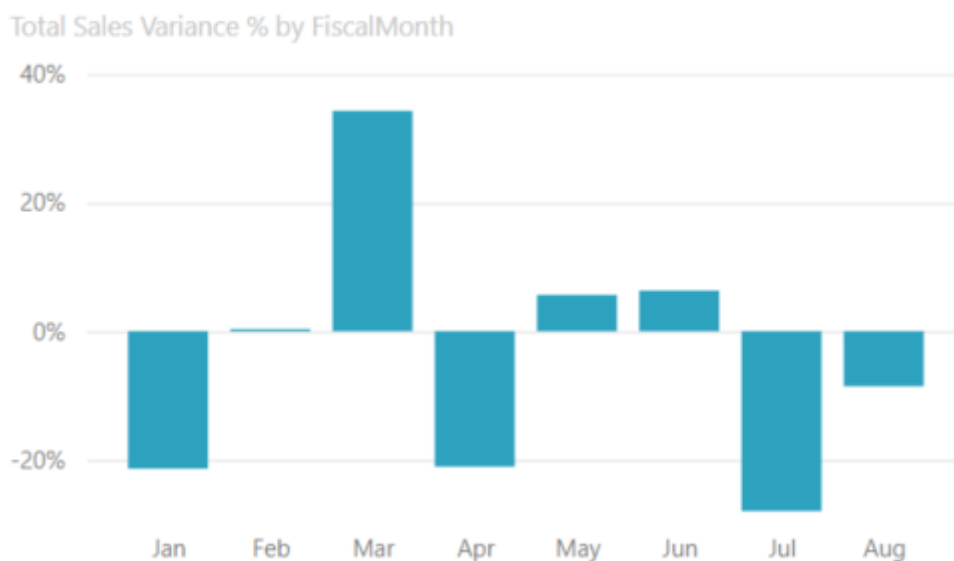
- Tip: Add conditional formatting to card background or data label to indicate status.



- **Column/Bar Charts (categorical comparison and ranking)**

- Use to compare categories (Products, Regions). Prefer horizontal bars for long labels.

- Example: Top 10 Products by Sales.
 - Visual: Bar chart with `DIM_Product[Product]` and `[Total Sales]`.
 - Configure: Visual filters > Top N = 10 by `[Total Sales]`; sort by `[Total Sales]` descending.
- Tip: Use data labels for clarity; avoid 3D effects and unnecessary gridlines.

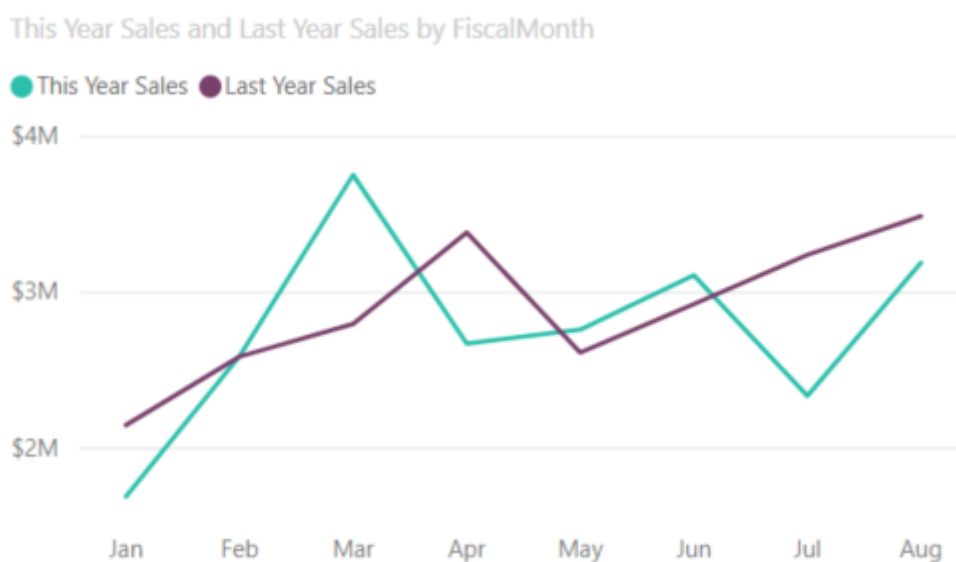


• Line Charts (trend over time)

- Use for continuous trends (daily, weekly, monthly).
- Example measures:

```
Sales MA(3) =
    VAR CurrDate = MAX( DIM_Date[Date] )
    VAR Window = DATESINPERIOD( DIM_Date[Date], CurrDate, -3,
    MONTH )
    RETURN AVERAGEX( Window, [Total Sales] )
```

- Tip: Use `Month Year` axis (e.g., `YYYY-MM`) to avoid sorting issues.



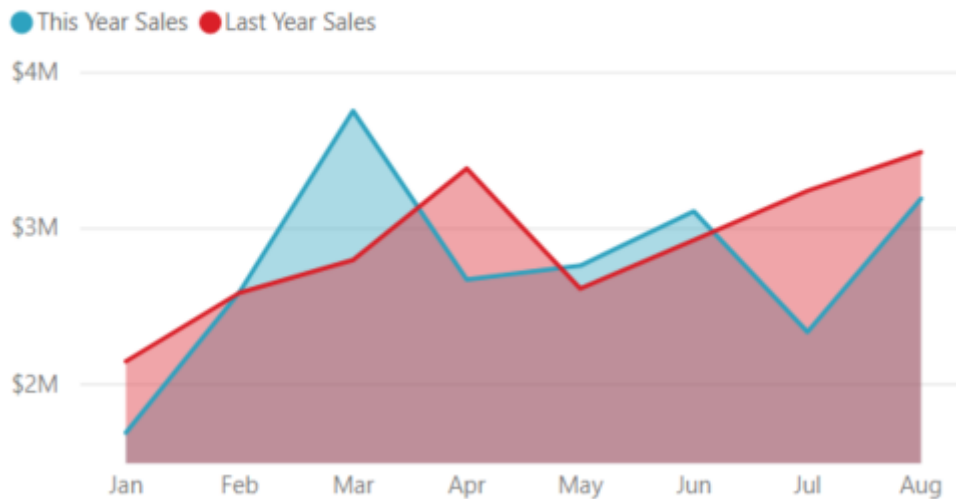
- **Area Charts (part-to-whole trend or cumulative)**

- Use for cumulative or stacked contributions over time.
- Example:

```
Sales YTD = TOTALYTD( [Total Sales], DIM_Date[Date] )
```

- Tip: Avoid 100% stacked area charts; they can mislead trend interpretation.

This Year Sales and Last Year Sales by FiscalMonth

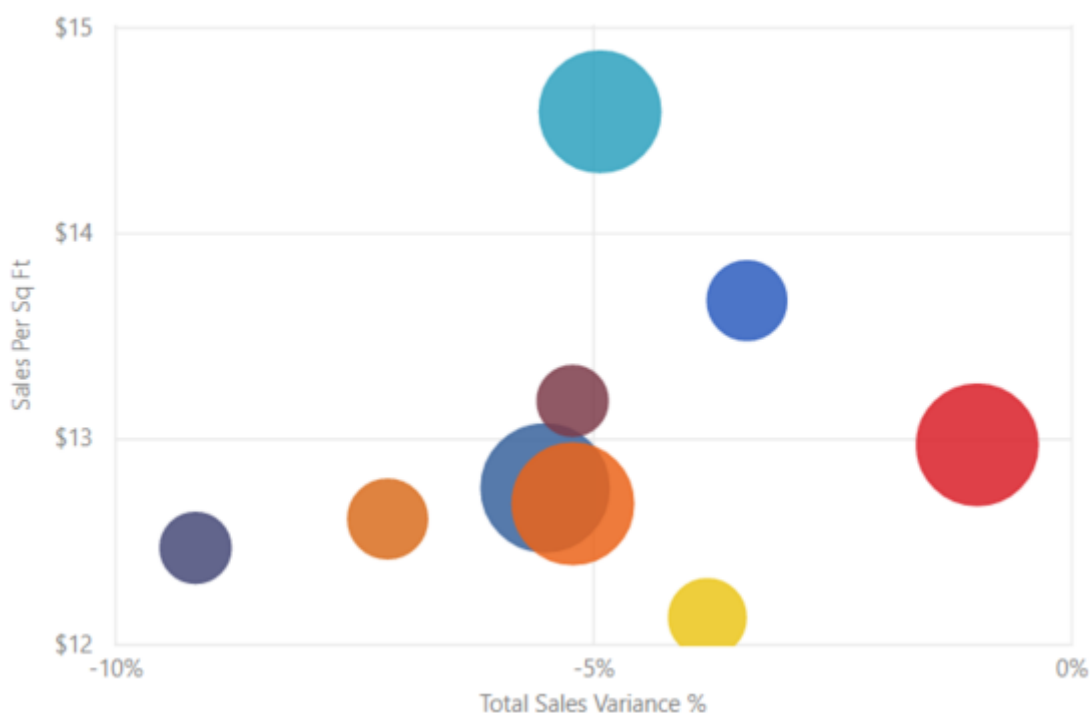


- **Scatter Charts (correlation)**

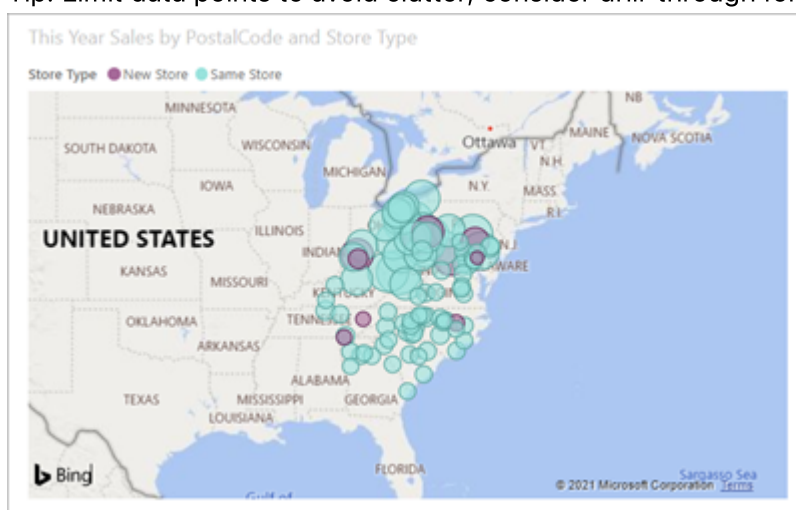
- Use to show relationships (e.g., Discount % vs Sales, Unit Price vs Quantity).
- Configure: X = [Discount %], Y = [Total Sales], Size = [Units], Category = DIM_Product[Category].

- Tip: Turn off Play Axis unless you really need time animation; it can be distracting and slow.

Total Sales Variance %, Sales Per Sq Ft and This Year Sales by District



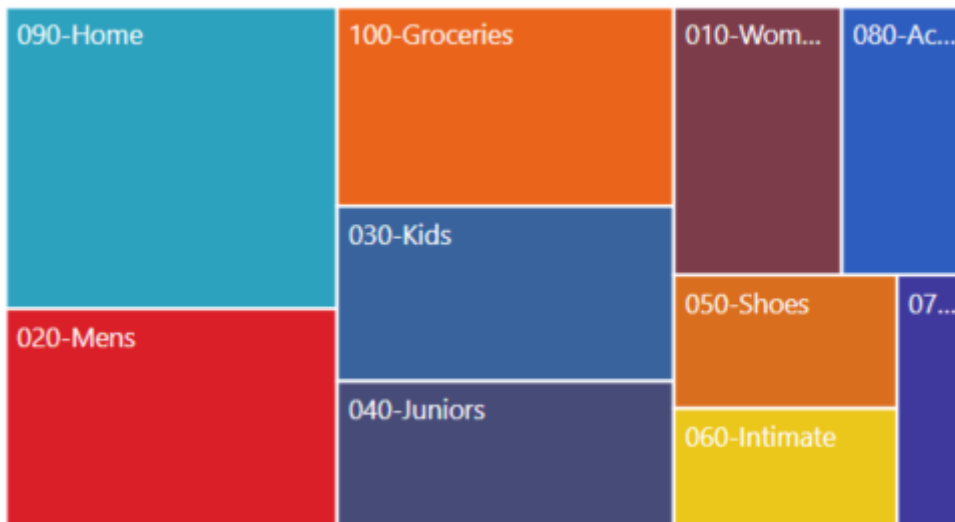
- Maps (geospatial aggregation)
 - Use for aggregated metrics by geography (Country, State). Avoid raw lat/long unless precise points are essential.
 - Example: Filled map with **Country** and **[Total Sales]**; ensure standardized country names and a single disambiguating column (ISO codes).
 - Tip: Limit data points to avoid clutter; consider drill-through for detailed views.



- TreeMap (part-to-whole with limited categories)
 - Use for hierarchical part-to-whole when there are few categories; avoid when many small segments exist.
 - Example: **Category** → **Subcategory** by **[Total Sales]**.

- Tip: Add data labels and tooltips for clarity.

Total Units Last Year by Category



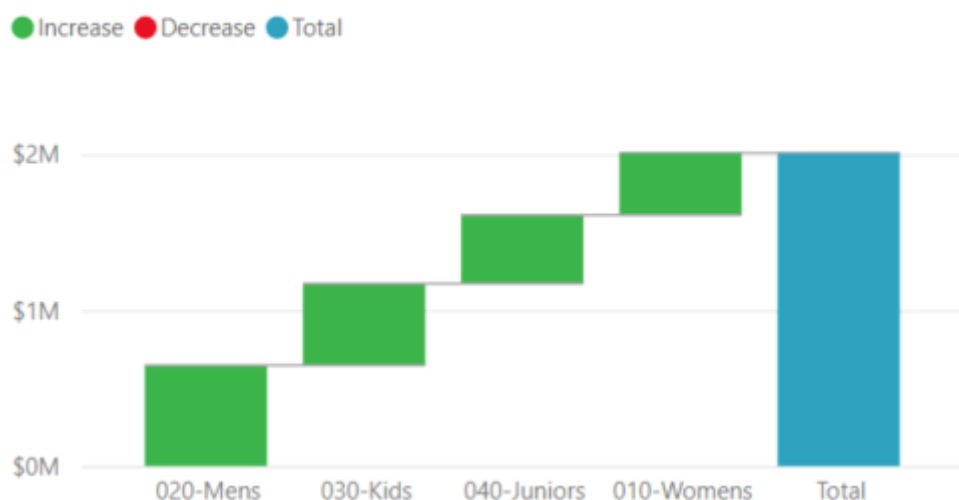
- Waterfall (variance analysis)

- Use to explain change from a starting value to an ending value by contributions.
- Example measures:

```
Sales LY = CALCULATE( [Total Sales], SAMEPERIODLASTYEAR(
  DIM_Date[Date] ) )
Sales Var = [Total Sales] - [Sales LY]
```

- Visual: Use categories like Region or Product to show positive/negative contributions.
- Tip: Clearly label totals and subtotals.

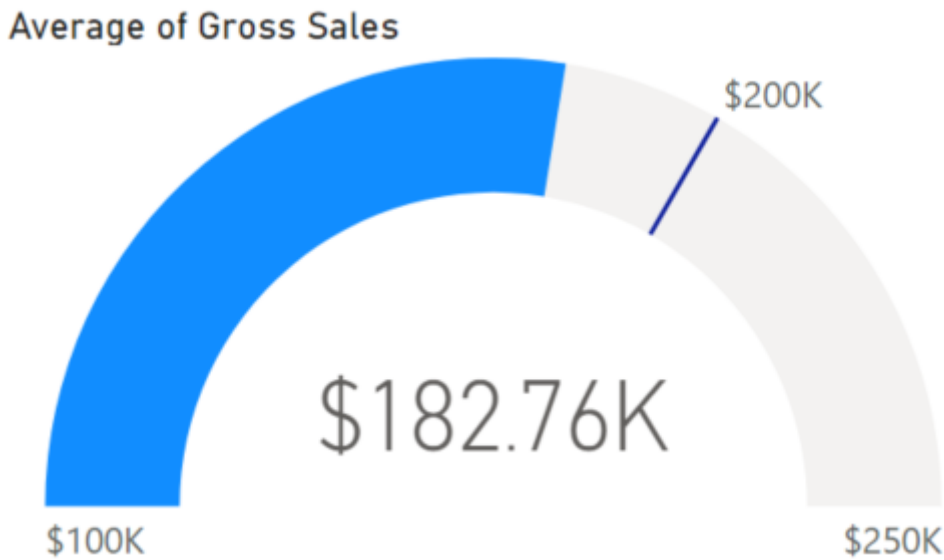
Total Units Last Year by Category



- Gauge (single KPI vs target)

- Use for a single metric against a specific target; avoid multiple gauges.
- Configure: Value = [On-time %], Target = 0.95 (or a measure), Min/Max = clear bounds.

- Tip: Use sparingly; consider cards or KPI visuals for clarity.



- Table vs Matrix (detail and pivoted breakdown)
 - Table: Use for detailed lists with row-level actions; limit columns, add search.
 - Matrix: Use for cross-tab breakdown (e.g., Region by Month with [Total Sales]). Enable subtotals where they aid interpretation.
 - Tips: Use conditional formatting (icons/bars), field formatting (thousand separators), and drill-down paths.

Drill on

Rows

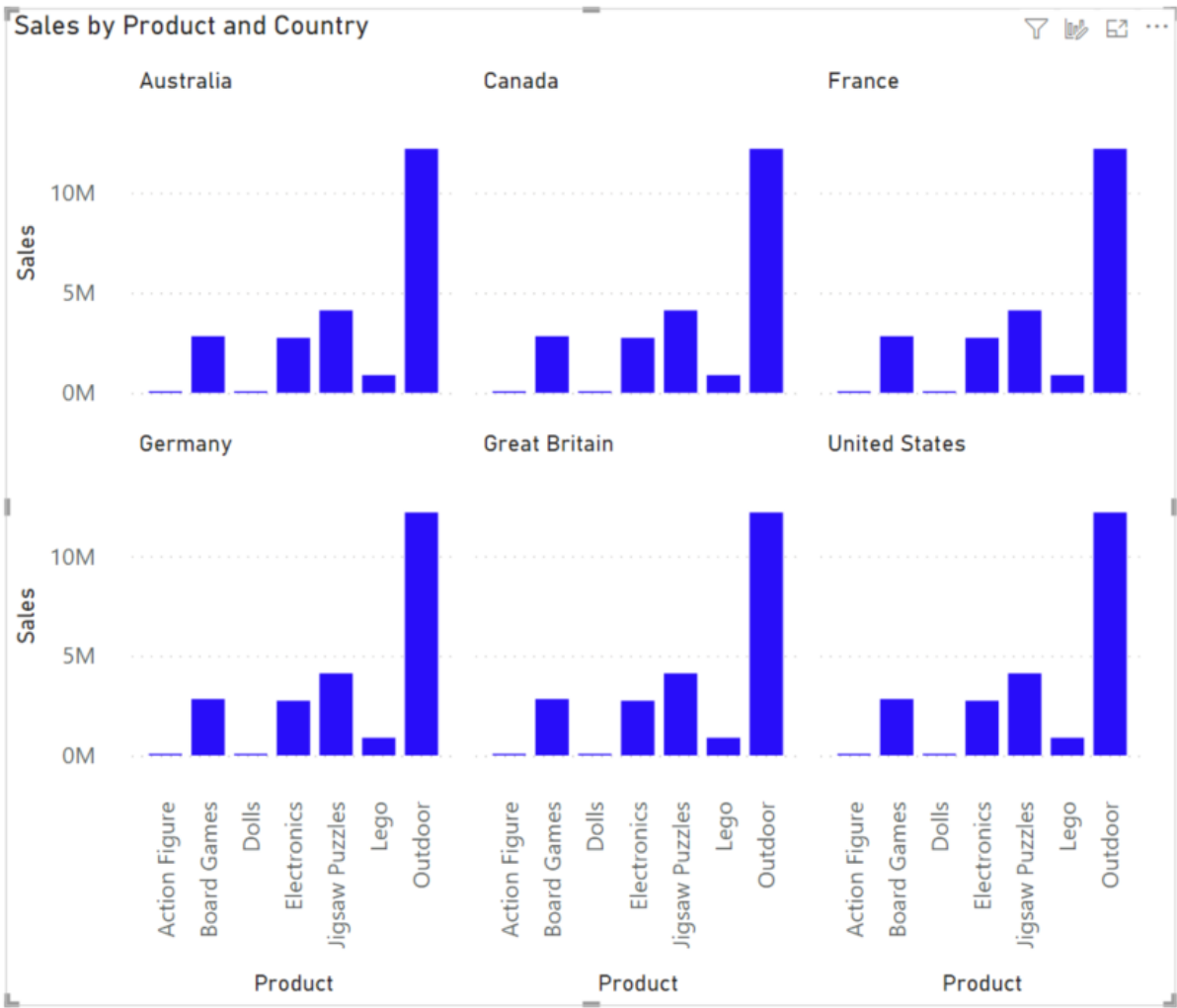
Region	Central		East		West		Total	
Sales Stage	Opportunity Count	Revenue	Opportunity Count	Revenue	Opportunity Count	Revenue	Opportunity Count	Revenue
Lead	102	\$507,574,417	114	\$473,887,837	52	\$256,159,114	268	\$1,237,621,368
Qualify	29	\$111,715,461	50	\$195,692,154	15	\$52,442,363	94	\$359,849,978
Solution	29	\$100,743,789	30	\$134,347,170	15	\$53,441,501	74	\$288,532,460
Proposal	14	\$46,722,869	13	\$59,970,924	10	\$43,032,669	37	\$149,726,462
Finalize	5	\$23,302,246	5	\$30,696,428	4	\$21,176,185	14	\$75,174,859
Total	179	\$790,058,782	212	\$894,594,513	96	\$426,251,832	487	\$2,110,905,127

- Decomposition Tree (guided root-cause breakdown)
 - Use to let users explore contributions across dimensions interactively.
 - Example: Analyze [Sales Var] by Region → Product → Customer Segment.
 - Tip: Requires measures; consider RLS implications and performance on large models.

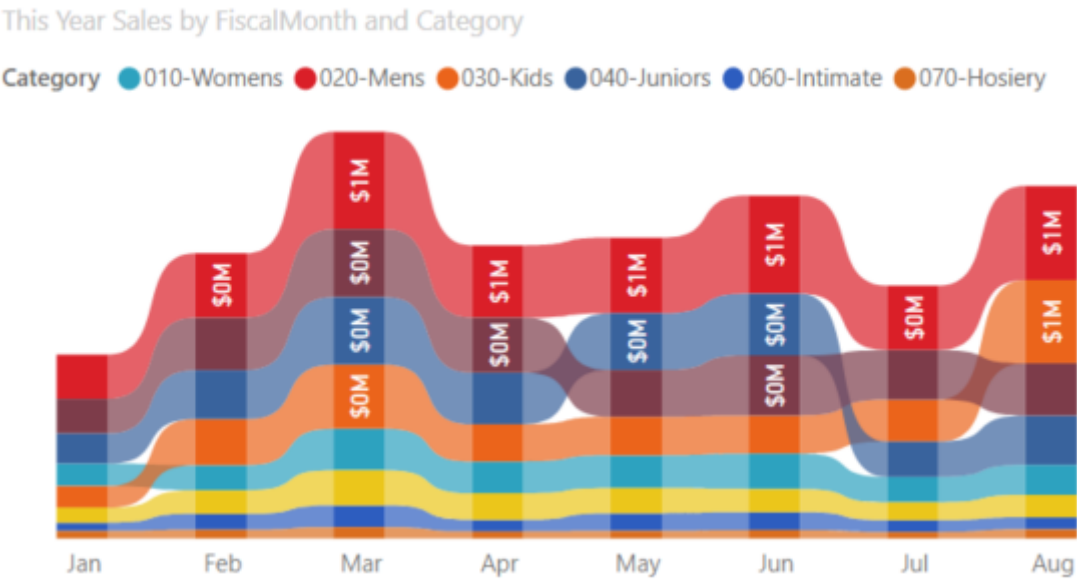


- Small Multiples (series comparison)

- Use to show the same visual repeated per category (e.g., monthly sales by Region).
- Tip: Limit the number of categories to preserve readability; prefer consistent axes.



- Ribbon Chart (rank changes over time)
 - Use to show ranking shifts (e.g., Segment rank per month by [Total Sales]).
 - Tip: Keep categories small (<= 5–6) to avoid clutter.



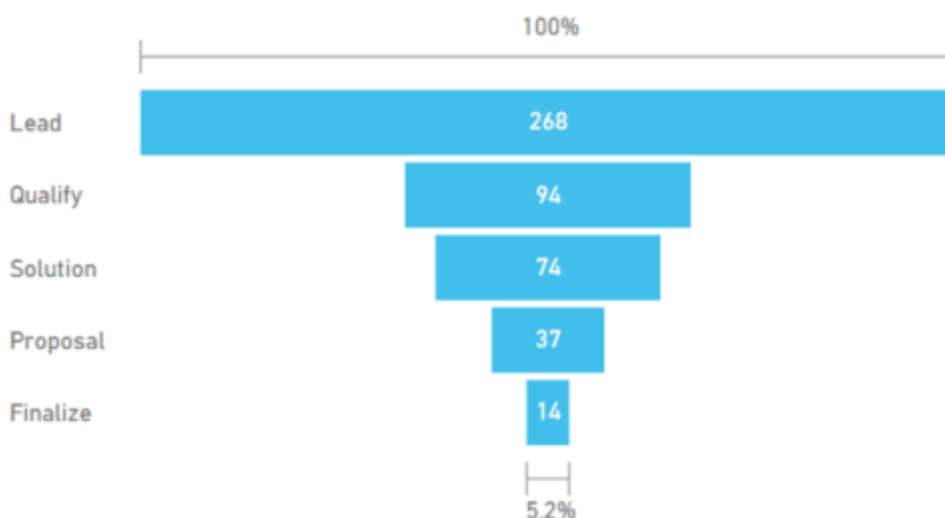
- Funnel (stage conversion)

- Use for linear processes (e.g., Leads → Opportunities → Wins) with decreasing counts.
- Example measures:

```
Leads = COUNTROWS( FACT_Pipeline )  
Opportunities = CALCULATE( COUNTROWS( FACT_Pipeline ),  
    FACT_Pipeline[Stage] = "Opportunity" )  
Wins = CALCULATE( COUNTROWS( FACT_Pipeline ),  
    FACT_Pipeline[Stage] = "Won" )  
Conversion % = DIVIDE( [Wins], [Leads] )
```

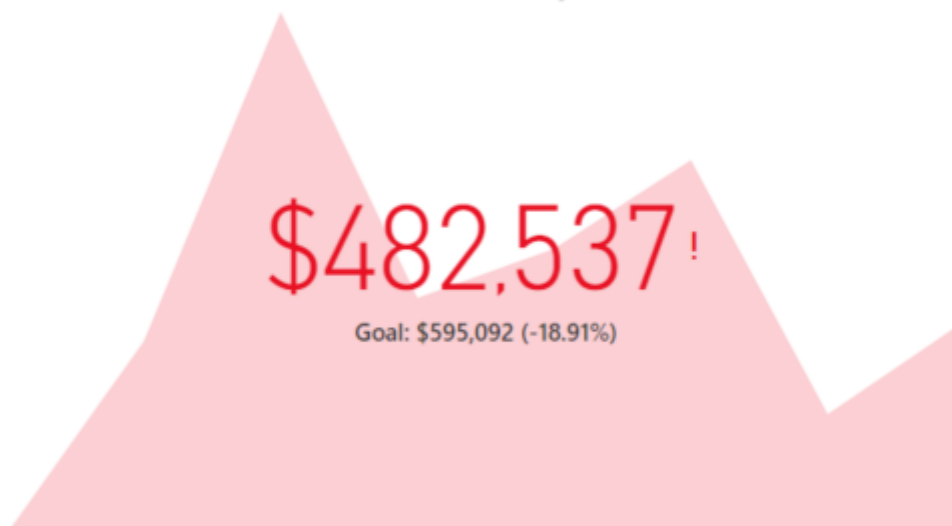
- Tip: Avoid using funnels for non-linear data or when stages can increase.

Opportunity Count by Sales Stage



- KPI Visual (target + trend)
 - Use when you need current value, target, and an indicator with optional trend line.
 - Configure: Value = [Total Sales], Target = [Sales Target], Trend axis = DIM_Date[Date].
 - Tip: Use sparingly; consider cards with trend lines as alternatives.

Total Units This Year and Total Units Last Year by Month

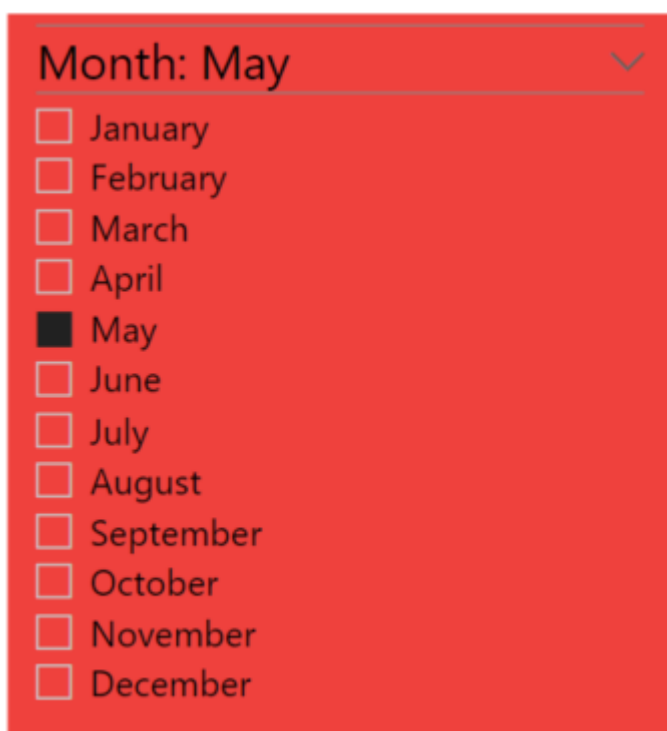


- **Slicers (filter controls)**

- Use to let users filter pages or synced pages by key dimensions (Date, Product, Region). Prefer Dropdown over List for long categories; avoid high-cardinality fields.
- Common setups:
 - Date: use **Between** or **Relative** slicer on **DIM_Date [Date]** from a marked Date table.
 - Category: Dropdown on **DIM_Product [Category]** with Search enabled.
 - Cross-page: Use the Sync slicers pane to reuse a single slicer across pages.
- Example helper measure (optional label):

```
Selected Category = SELECTEDVALUE( DIM_Product[Category], "(All)" )
```

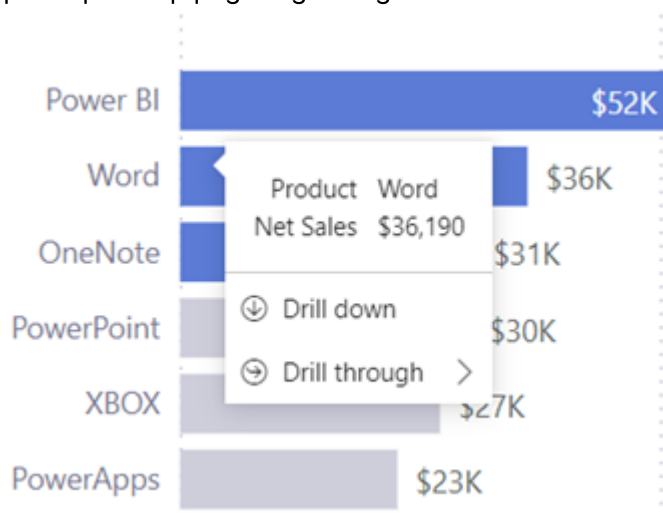
- Tips: Enable Query reduction Apply buttons for slicers; limit interactions (Format > Edit interactions) to reduce backend queries; avoid cascading slicers on high-cardinality columns.



- **Tooltips (on-demand detail and micro-charts)**

- Use standard or report page tooltips to show more context (e.g., 12-month sparkline, segment mix) without loading heavy visuals on the main page.

- Tip: Keep tooltip pages lightweight.



Frequent Asked Questions

1. When should I use a star schema instead of a snowflake?

- Prefer a star schema in the model: it simplifies joins, improves performance, and eases maintenance. Keep snowflaking (normalization) upstream in your warehouse/ETL; flatten to dimensions for the model. See Data Modeling for examples.

2. Import vs DirectQuery vs Composite — how do I choose?

- Import is the default for speed and full DAX features. Use DirectQuery when data volume or latency requirements make import impractical. Composite models let you combine Import dimensions with a DirectQuery fact, often with Aggregations to speed common queries.

3. Measures vs calculated columns — when to use each?

- Use measures for aggregations that respect filter context (dynamic at query time). Use calculated columns for row-level attributes needed for relationships, sorting, or slicers. Avoid columns for values that could be computed as a measure (e.g., row totals).

4. How do I set up a proper calendar and avoid Auto Date/Time bloat?

- Create a reusable Date table (DAX `CALENDAR` or `CALENDARAUTO` with added attributes), mark it as a Date table, and disable Auto Date/Time in Options to prevent hidden tables. Use that single Date table for all time intelligence.

5. Why use `DIVIDE()` instead of the `/` operator?

- `DIVIDE(numer, denom, alternate)` safely handles divide-by-zero and blank denominators and yields clearer intent. It avoids error branches and lets you set a default result.

6. `COUNTROWS` vs `COUNT` — which is right for me?

- Use `COUNTROWS(table)` to count rows (ignores column blank issues). `COUNT(column)` counts non-blank values in a column and can be misleading if blanks exist. Prefer `COUNTROWS` for row counts.

7. VALUES vs DISTINCT — what's the difference?

- Both return unique values, but **VALUES ()** may include the Power BI-added blank row due to referential integrity gaps; **DISTINCT ()** only includes blanks present in source data. Choose one consistently across your model and visuals.

8. When are bi-directional or many-to-many relationships appropriate?

- Avoid them by default due to performance and ambiguity. Prefer single-direction relationships and activate alternate paths in measures (**USERELATIONSHIP**, **CROSSFILTER**) when needed for a specific calculation.

9. How do I keep slicers performant on large datasets?

- Avoid high-cardinality fields, use Dropdown style with Search, enable Query reduction Apply buttons, sync a single slicer across pages, and limit unnecessary visual interactions so slicer changes don't trigger all visuals.

10. What's the simplest way to set up Incremental Refresh?

- Add **RangeStart** and **RangeEnd** DateTime parameters, filter your fact table on **[Date] >= RangeStart and < RangeEnd** in Power Query, then configure the Incremental refresh policy in Desktop (store/refresh windows) before publishing. This reduces refresh time and service load.